



Generalized Gumbel-Softmax gradient estimator for generic discrete random variables

Weonyoung Joo ^a, Dongjun Kim ^b, Seungjae Shin ^c, Il-Chul Moon ^d

^a Department of Statistics, Ewha Womans University, Seoul, Republic of Korea

^b Department of Computer Science, Stanford University, CA, United States

^c Qualcomm AI Research, Seoul, Republic of Korea

^d Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

ARTICLE INFO

Editor: Jiwen Lu

Keywords:

Reparameterization trick
Stochastic gradient estimator
Gumbel-softmax trick
Discrete random variable
Variational autoencoder
Deep generative model

ABSTRACT

Estimating the gradients of stochastic nodes in stochastic computational graphs is one of the crucial research questions in the deep generative modeling community, which enables gradient descent optimization on neural network parameters. Stochastic gradient estimators of discrete random variables, such as the Gumbel-Softmax reparameterization trick for Bernoulli and categorical distributions, are widely explored. Meanwhile, other discrete distribution cases, such as the Poisson, geometric, binomial, multinomial, negative binomial, etc., have not been explored. This paper proposes a generalized version of the Gumbel-Softmax stochastic gradient estimator. The proposed method is able to reparameterize generic discrete distributions, not restricted to the Bernoulli and the categorical, and it enables learning on large-scale stochastic computational graphs with discrete random nodes. Our experiments consist of (1) synthetic examples and applications on variational autoencoders, which show the efficacy of our methods; and (2) topic models, which demonstrate the value of the proposed estimation in practice.

1. Introduction

Stochastic computational graphs, a sub-type of deep neural networks which include variational autoencoders [1], are widely used for probabilistic modeling, latent modeling, representation learning, and generating data in the deep generative model (DGM) society [2,3]. Optimizing such network parameters through back-propagating gradients requires an estimation of the gradient values. However, the stochasticity requires the computation of expectation, which differentiates this problem from the deterministic gradient of ordinary neural networks. Regarding such a perspective, there are two common ways of estimating the gradients from stochastic nodes: score function (SF) methods and reparameterization methods. The SF-based estimators tend to result in unbiased gradients with high variances; hence, the SF-based estimators aim to reduce the variances of gradients for stable and fast optimizations. Meanwhile, the reparameterization estimators result in biased gradients with low variances [4], but they require the *differentiable non-centered parameterization* [5] of random variables.

For continuous random variables such as the Gaussian, the reparameterization estimators are widely utilized due to the nature of the differentiability of the typical continuous distributions [1,6,7].

Nowadays, it is feasible to estimate gradients for continuous cases with automatic differentiation [8,9] in TensorFlow [10] or PyTorch [11]. Meanwhile, for the discrete random variables which follow Bernoulli or categorical distributions, the SF-based methods are widely explored since the reparameterization method cannot directly work due to the non-differentiability. Alternatively, the Gumbel-Softmax trick [12,13] overcomes this difficulty through the reparameterization with continuous relaxation of one-hot selection values. Also, a line of work [14,15] utilizes both SF-based and reparameterization methods.

While the Bernoulli and the categorical cases have been studied deeply, the gradient estimators for other discrete distributions have hardly been explored. Several gradient estimators are specialized in discrete distributions on combinatorial spaces, such as k -hot vector, permutation, and spanning trees on graphs [16,17]. However, other renowned discrete distribution cases, such as Poisson, binomial, multinomial, geometric, negative binomial distributions, etc., have not been explored, which we mainly focus on in this paper. Prior works on probabilistic graphical models, such as Ranganath et al. [18,19], adopted Poisson latent variables for latent counting. Another line of work [20]

* Corresponding author.

E-mail addresses: weonyoungjoo@ewha.ac.kr (W. Joo), dongjun@stanford.edu (D. Kim), sseungja@qti.qualcomm.com (S. Shin), icmoon@kaist.ac.kr (I.-C. Moon).

<https://doi.org/10.1016/j.patrec.2025.05.024>

Received 23 February 2023; Received in revised form 28 May 2025; Accepted 29 May 2025

Available online 13 June 2025

0167-8655/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

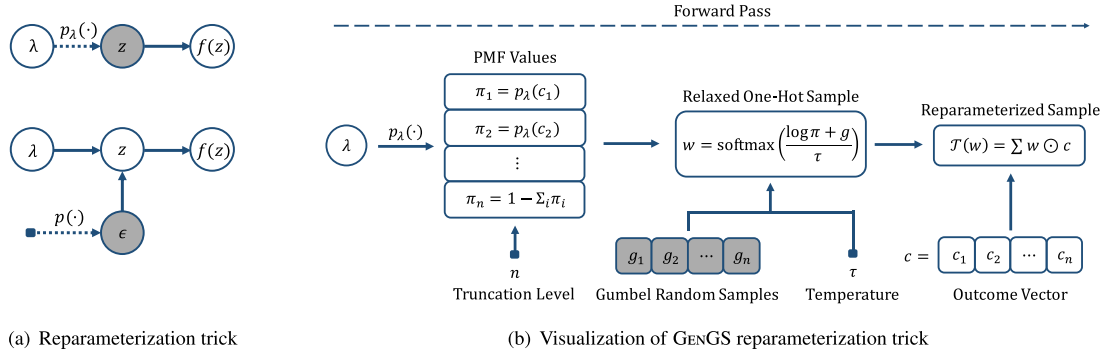


Fig. 1. Visualization of reparameterization tricks. The shaded nodes indicate random nodes, and the dotted lines denote sampling processes. The auxiliary random variables enable the back-propagation flow of the gradients.

utilized the Gaussian approximation on the Poisson to count the number of words in deep generative modeling, which can be a poor approximation when the rate parameter is small. Regarding those perspectives, the stochastic gradient estimator for the discrete distribution needs to be studied further to extend the choice of prior assumptions.

This paper proposes a generalized version of the Gumbel-Softmax trick, which can reparameterize the broader ranges of discrete distributions, not limited to the Bernoulli and the categorical. The proposed Generalized Gumbel-Softmax (GENGS) is probabilistically grounded, and it utilizes (1) a truncation to finitize the infinite support of the discrete distributions; and (2) a transformation that enables the generalization of the Gumbel-Softmax trick. Through the truncation, GENGS provides a posterior inference procedure that can be either explicit or implicit as a practical implementation. The GENGS reparameterization trick results in low bias and variance compared to other stochastic gradient estimators, and it leads to better optimum points in our experiments. Finally, the topic model application demonstrates the usability of the proposed stochastic gradient estimator.

2. Preliminary

2.1. Back-propagation through stochastic nodes

Assume that there is an intermediate stochastic node or a latent variable $z \sim p(z|\lambda)$ in the stochastic computational graphs, where the distribution depends on the parent node λ as a distribution parameter which we want to optimize. The goal is to optimize the objective function

$$\mathcal{L}(\lambda, \eta) = \mathbb{E}_{z \sim p(z|\lambda)} [f_\eta(z)], \quad (1)$$

where f_η is a differentiable function with respect to other model parameter η . To optimize the objective function with respect to the parameter λ through the gradient methods, we need to compute $\nabla_\lambda \mathcal{L}(\lambda, \eta) = \nabla_\lambda \mathbb{E}_{z \sim p(z|\lambda)} [f_\eta(z)]$, which is intractable with its original form.

SF-based estimator utilizes a score function $\nabla_\lambda \log p(z|\lambda)$ by utilizing a log-derivative trick to compute the gradient $\nabla_\lambda \mathcal{L}(\lambda, \eta)$ [21]. Then, the gradient $\nabla_\lambda \mathcal{L}(\lambda, \eta)$ can be derived as below:

$$\nabla_\lambda \mathcal{L}(\lambda, \eta) = \mathbb{E}_{z \sim p(z|\lambda)} [f_\eta(z) \nabla_\lambda \log p(z|\lambda)]. \quad (2)$$

The SF methods compute exact gradients due to the unbiasedness of the Monte Carlo property. However, those methods suffer from a high variance of gradients, which results in the slow and unstable convergence of the loss function during the training. To reduce the variance of gradients, the control variate methods [22,23] or the Rao-Blackwellization [24,25] are widely used.

Reparameterization trick, illustrated in Fig. 1(a), introduces an auxiliary variable $\epsilon \sim p(\epsilon)$, which takes over randomness of the latent

variable z , to compute $\nabla_\lambda \mathcal{L}(\lambda, \eta)$. Then, the sampled value z can be reformulated as $z = g(\lambda, \epsilon)$, with a *deterministic* and *differentiable* function g in terms of λ . Here, the gradient $\nabla_\lambda \mathcal{L}(\lambda, \eta)$ is derived as below:

$$\nabla_\lambda \mathcal{L}(\lambda, \eta) = \mathbb{E}_{\epsilon \sim p(\epsilon)} [\nabla_z f_\eta(z) \nabla_\lambda z]. \quad (3)$$

Eq. (3) is now computable, however, the differentiability requires the continuity of the random variable z , so the distribution of z is limited to the continuous distributions.

2.2. Variational autoencoders via reparameterization tricks

Variational autoencoders (VAEs) are one of the most popular deep neural networks which adopt the objective function of the form in Eq. (1). For observation x and its latent variable z , VAE assumes generative process $p_\theta(x|z)$, namely decoder with neural network parameter θ , with prior assumption $p(z)$ on the latent variable. To infer the latent variable z , VAE introduces a variational distribution or approximate posterior $q_\phi(z|x)$, called encoder parameterized by ϕ . Due to the intractability of log-evidence $\log p(x)$, VAE instead optimizes its lower bound, called evidence lower bound (ELBO)

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p_\theta(z)), \quad (4)$$

and the optimization of encoder parameter ϕ can be problematic as we discussed in Section 2.1.

VAEs employ the reparameterization trick to optimize the neural network parameters, and there is a line of work which utilizes diverse latent priors to VAEs along with the reparameterization tricks tailored to each prior. The regular VAE [1] assumes the Gaussian posterior $\mathcal{N}(\mu, \Sigma)$ with simple reparameterization trick, where $z = \mu + \epsilon \times \Sigma$ with an alternative sampling $\epsilon \sim \mathcal{N}(0, I)$ instead of direct sampling $z \sim \mathcal{N}(\mu, \Sigma)$. Hence, it can be viewed as a simple change-of-variable technique. To utilize Dirichlet distribution to the latent variable in VAE, the authors of Joo et al. [7] first decompose Dirichlet random variable into gamma random variables, and then approximate the inverse cumulative distribution function of the gamma distribution to alternatively sample cumulative distribution function from uniform distribution $\mathcal{U}[0, 1]$. For non-parametric latent dimension of VAE with stick-breaking prior, the authors of Nalisnick and Smyth [6] approximate beta random variable included in the stick-breaking process with Kumaraswamy random variable, which allows the reparameterization with tractable inverse cumulative distribution function. The detailed reparameterization tricks and characteristics of each VAE can be found in the respective literature.

2.3. Gumbel tricks for categorical random variable

While continuous random variables can be reparameterized with usual change-of-variable techniques, discrete random variables suffer from differentiability due to the nature of discrete probability support. To utilize the *differentiable* reparameterization trick on categorical random variables, continuous relaxation can be applied. A *Gumbel-Softmax* (GS) trick [12,13] is an approximation of a *Gumbel-Max* (GM) trick, which are alternatives to a one-hot categorical sampling. The categorical random variable $Z \sim \text{Cat}(\pi)$, where π in the $(n-1)$ -simplex Δ^{n-1} , can be reparameterized by the GM trick: (1) draw $u_j \sim \text{Uniform}(0, 1)$ to generate an auxiliary Gumbel sample $g_j = -\log(-\log u_j)$ for $j = 1, \dots, n$; and (2) compute $k = \arg\max_{j=1}^n [g_j + \log \pi_j]$. This procedure generates a one-hot sample z such that $z_k = 1$ with $P(Z_k = 1) = \pi_k$ and zeros in other entries. Instead of the $\arg\max$ in the GM, the GS utilizes the softmax with a temperature $\tau > 0$, i.e., $z \approx \text{softmax}(\frac{g + \log \pi}{\tau})$. This substitution relaxes the discreteness of the categorical random variable to the one-hot-like value in the continuous domain. In other words, if we denote $\text{GM}(\pi)$ and $\text{GS}(\pi, \tau)$ as the random variables generated by the GM trick and the GS trick, respectively, with the same Gumbel random variable, then the continuous sample generated from $\text{GS}(\pi, \tau)$ converges to the discrete sample generated from $\text{GM}(\pi)$ as $\tau \rightarrow 0$.

3. Methodology

3.1. Problem setting

We begin an explanation on GENGS with discrete distribution having finite support. Assume that a random variable Z follows a discrete distribution $Q(\lambda)$ with explicit probability mass function (PMF) and finite support $C = \{c_1, \dots, c_n\}$. Next, define an outcome vector $c = (c_1, \dots, c_n)$ and the corresponding PMF vector $\pi = (\pi_1, \dots, \pi_n)$ in the same index order. Here, each sample c_k can be either scalar or vector (or even matrix) by choice of distribution Q . Finally, for $w = (w_1, \dots, w_n) \in \Delta^{n-1}$, introduce a transformation $\mathcal{T}$ as follows:

$$\mathcal{T}(w) = \sum_{k=1}^n w_k c_k := \sum w \odot c. \quad (5)$$

3.2. Sampling through generalized Gumbel trick

We claim that the sampling process of $Z \sim Q(\lambda)$ can be derived by the one-hot categorical selection process, i.e., the GM process. Suppose that we draw a sample $Z = c_m \in C$, then the sampled value c_m can be alternatively selected as follows. The one-hot indicating vector of sample c_m can be regarded as $w = \text{one_hot}(c_m|c)$ because we fix the index order of the outcome vector c and its corresponding PMF vector π . Note that c_m from $Q(\lambda)$ and w from $\text{GM}(\pi)$ have the same sampling probability, and the transformation $\mathcal{T}$ and the one_hot function define the one-to-one correspondence of the two sample spaces. Specifically, the conversion of $w = \text{one_hot}(c_m|c)$ to c_m can be done by the deterministic and differentiable transformation $\mathcal{T}$ in Eq. (5). Therefore, we can reparameterize the original sample value c_m as $\mathcal{T}(\text{GM}(\pi))$ by replacing the randomness of Z from λ with the auxiliary Gumbel random variable in $\text{GM}(\pi)$. Proposition 1 states the reparameterization trick, and we substitute the proof with the explanation above.

Proposition 1. For the transformation $\mathcal{T}$, if a discrete random variable $Z \sim Q(\lambda)$ has finite support with known PMF vector π , Z can be reparameterized by $\mathcal{T}(\text{GM}(\pi))$ where $\pi_k = P(Z = c_k)$.

3.3. Relaxation of one-hot sample

Replacing the $\arg\max$ in the GM trick with the softmax enables differentiation, which is a key point of the GS reparameterization. In Proposition 2, we claim that the same argument still holds under the transformation $\mathcal{T}$: if $W \sim \text{GS}(\pi, \tau)$ where π is a PMF vector of discrete random variable $Z \sim Q(\lambda)$ with finite support, then Z can be continuously relaxed and reparameterized as $\mathcal{T}(\text{GS}(\pi, \tau))$, which we define as $\text{GENGS}(\pi, \tau)$.

Proposition 2. For the transformation $\mathcal{T}$ and a categorical parameter $\pi \in \Delta^{n-1}$, the convergence property of GS to GM still holds under the transformation $\mathcal{T}$, i.e., $\text{GS}(\pi, \tau) \rightarrow \text{GM}(\pi)$ as $\tau \rightarrow 0$ implies $\text{GENGS}(\pi, \tau) \rightarrow \mathcal{T}(\text{GM}(\pi))$ as $\tau \rightarrow 0$.

Proof. Define f_M and f_S^τ be the GM and GS reparameterization with a temperature $\tau > 0$, respectively, where the both take the categorical parameter $\pi \in \Delta^{n-1}$ and a Gumbel sample as inputs. For the Gumbel sample g , if we assume $g_m + \log \pi_m > g_j + \log \pi_j$ for all $j \neq m$, then $\mathcal{T}(f_M(\pi, g)) = c_m$ holds. The statement $\text{GS}(\pi, \tau) \rightarrow \text{GM}(\pi)$ as $\tau \rightarrow 0$ implies $f_S^\tau(\pi, g) \rightarrow f_M(\pi, g)$, i.e.,

$$\begin{aligned} [f_S^\tau(\pi, g)]_j &= \frac{\exp(\frac{g_j + \log \pi_j}{\tau})}{\sum_{k=1}^n \exp(\frac{g_k + \log \pi_k}{\tau})} \\ &\rightarrow \begin{cases} 1 & \text{if } j = m \\ 0 & \text{if } j \neq m \end{cases} \quad \text{as } \tau \rightarrow 0. \end{aligned}$$

As a consequence, we have the following:

$$\begin{aligned} [f_S^\tau(\pi, g)]_j \times c_j &= \left[\sum_{k=1}^n \frac{\exp(\frac{g_k + \log \pi_k}{\tau})}{\sum_{k=1}^n \exp(\frac{g_k + \log \pi_k}{\tau})} \right]_j \times c_j \\ &\rightarrow \begin{cases} c_m & \text{if } j = m \\ 0 & \text{if } j \neq m \end{cases} \quad \text{as } \tau \rightarrow 0. \end{aligned}$$

Hence, by taking the summation, the following holds:

$$\mathcal{T}(f_S^\tau(\pi, g)) = \sum_{k=1}^n [\tilde{e}_m]_k \cdot c_k \rightarrow c_m = \mathcal{T}(f_M(\pi, g)). \quad \square$$

3.4. Finitizing support by truncation

In Section 3.1, we assumed that the support of the distribution to be finite. Without the finite assumption, the Gumbel tricks cannot be applied since it requires a limited number of categories. Therefore, for the discrete distributions with infinite support, we truncate the tails of the distributions to approximate the original distribution, and we now can extend GENGS to the discrete distribution with infinite support. Definition 3 defines truncated distribution of non-negative discrete random variable.

Definition 3. For a non-negative discrete random variable $X \sim D(\lambda)$, define $Z_n = X$ if $X \leq n-1$, and $Z_n = n-1$ if $X > n$. The random variable Z_n is said to follow a *truncated discrete distribution* $\text{TD}(\lambda, R)$ with a parameter λ and a truncation range $R = [0, n)$. Alternatively, we write as truncation level $R = n$ if the left truncation is at zero in the non-negative case.

Definition 3 not only finitizes the support but also provides the modification of the PMF: $\pi_k = P(Z_n = c_k) = P(X = c_k)$ for $k = 1, \dots, n-1$, and $\pi_n = P(Z_n = c_n) = 1 - \sum_{k=1}^{n-1} \pi_k$. For the truncated discrete random variable $Z_n \sim \text{TD}(\lambda, R)$ having an outcome vector $c = (c_1, \dots, c_n)$, the ordering of c_k is not significant as long as the index of the outcome vector c and the PMF vector π are aligned. Proposition 4 provides a theoretical guarantee that Z_n approximates X with convergence of random variables as the truncation range widens. Hence, the truncation enables GENGS to be extendedly applicable to the generic distributions, such as the Poisson, geometric, negative binomial, etc.

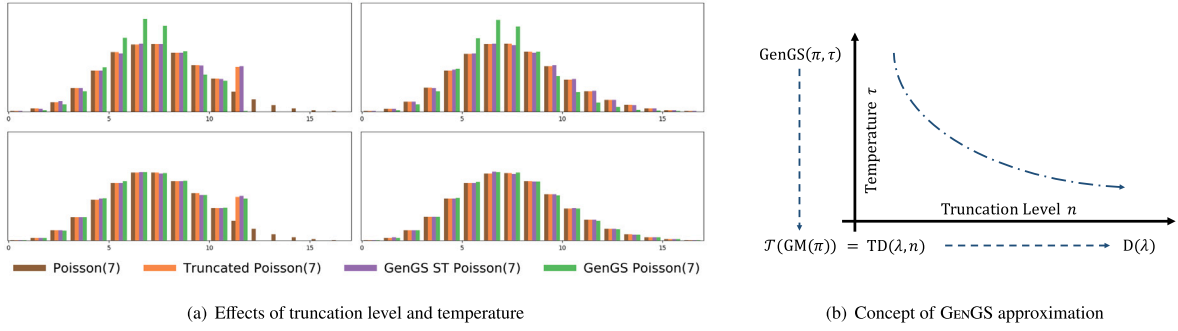


Fig. 2. (a) Approximation of GENGS in terms of choices of the truncation level n and the temperature τ in Poisson(7). As sub-figures go from left to right, the truncation level grows. Hence, the popped-out sticks, implying the remaining probability of the right side, disappears if the truncation level is large enough. As sub-figures go from top to bottom, the temperature decreases, and the PMF of truncated discrete distributions and the original distributions become similar. (b) On the y -axis, as temperature $\tau \downarrow 0$, $\text{GENGS}(\pi, \tau) \rightarrow \text{TD}(\lambda, n)$, where π is a computed PMF vector of $\text{TD}(\lambda, n)$, according to Proposition 2. $\text{TD}(\lambda, n)$ can be reparameterized by the GM trick with a transformation $\mathcal{T}$ as in Proposition 1. On the x -axis, $\text{TD}(\lambda, n) \rightarrow D(\lambda)$ as truncation level $n \uparrow \infty$, according to Proposition 4.

Algorithm 1 Alternative sampling of GENGS with explicit PMF.

```

1: Input: Distribution  $D(\lambda)$  with PMF  $p_\lambda(\cdot)$ , finite possible outcome vector  $c$ 
   within truncation range  $R$ , temperature  $\tau$ 
2:  $\pi_k = p_\lambda(c_k)$  for  $k = 1$  to  $n - 1$ 
3:  $\pi_n = 1 - \sum_{k=1}^{n-1} \pi_k$ 
4: Draw Gumbel samples  $g_1, \dots, g_n$ 
5: Compute relaxed one-hot sample  $w$  with  $\text{GS}(\pi, \tau)$ 
6: Compute  $z = \mathcal{T}(w) = \sum w \odot c$ 
7: return  $z$ 

```

Proposition 4. Z_n converges to X almost surely as $n \rightarrow \infty$.

Proof. Note that $\{\omega \in \Omega | Z_n(\omega) = X(\omega)\} = \{\omega \in \Omega | X(\omega) < n\}$ for sample space Ω . Then, we have the following:

$$\begin{aligned}
 P(\lim_{n \rightarrow \infty} Z_n = X) &= P(\{\omega \in \Omega | \lim_{n \rightarrow \infty} Z_n(\omega) = X(\omega)\}) \\
 &= P(\{\omega \in \Omega | \lim_{n \rightarrow \infty} Z_n(\omega) = \lim_{n \rightarrow \infty} X(\omega) < \lim_{n \rightarrow \infty} n\}) \\
 &= P(\{\omega \in \Omega | X(\omega) < \infty\}) \\
 &= P(X < \infty) \\
 &= 1
 \end{aligned}$$

since X is a non-negative discrete random variable. $\square$

Similarly, Definition 3 and Proposition 4 can be generalized to general discrete distribution without the non-negative assumption as Proposition 5, which we omit the proof.

Proposition 5. For a discrete random variable $X \sim D(\lambda)$, define (i) $Z_{m,n} = X$ if $m < X < n$; (ii) $Z_{m,n} = n - 1$ if $X \geq n$; and (iii) $Z_{m,n} = m + 1$ if $X \leq m$. Then, $Z_{m,n}$ converges to X almost surely as $m \rightarrow -\infty$ and $n \rightarrow \infty$.

4. Hyper-parameters & variations of GENGS

Fig. 1(b) illustrates the full steps of the GENGS trick, and Algorithm 1 shows the alternative sampling process of GENGS. The remainder of this section describes the hyper-parameter and the variations of the proposed GENGS. Fig. 2 describes how the GENGS trick approximates the original distribution by adjusting the temperature and truncation range, which are the two key hyper-parameters of the proposed GENGS.

4.1. Hyper-parameters

Temperature. Since GENGS utilizes the GS, GENGS inherits the temperature as a hyper-parameter. The decrement of temperature τ leads to

Algorithm 2 GENGS NP: GENGS with non-parametric truncation range.

```

1: Input: Distribution  $D(\lambda)$  with and PMF  $p_\lambda(\cdot)$ , temperature  $\tau$ , threshold  $\eta \lesssim 1$ 
2:  $k, \text{prob\_sum} = 0, 0$ 
3:  $\text{pi}, c = \text{list}(), \text{list}()$ 
4: while  $\text{prob\_sum} < \eta$  do
5:    $\text{pi.append}(p_\lambda(k)), c.append(k)$ 
6:    $\text{prob\_sum}, k += p_\lambda(k), 1$ 
7:  $\text{pi.append}(1 - \text{sum}(\text{pi})), c.append(k)$ 
8: return  $\text{pi}, c$ 
9: Apply GENGS with  $\text{pi}$  and  $\tau$ 

```

a distribution closer to the original distribution. However, the initially small τ leads to high bias and variance of gradients, which becomes problematic at the learning stage on π . Hence, the annealing of τ from a large value to a small value is necessary to provide a learning chance of π .

Truncation range. The truncated discrete distribution becomes closer to the original distribution as we widen the truncation range R , and the choice of truncation range is crucial in terms of covering many probable samples or the modes of the distribution. Hence, we can set the truncation range to cover most of the support through a certain threshold as a hyper-parameter. Also, for the popular discrete distributions such as Poisson, geometric, negative binomial, etc., which have a single mode in the PMF, we can use the mode as a criterion. Alternatively, we can also set non-parametric truncation range with a threshold, as in Algorithm 2.

4.2. GENGS variants

Implicit inference. The stochastic gradient estimators or the alternative sampling with the reparameterization tricks are widely utilized for the sampling from some distribution after inferring the distribution parameter. In such cases, there is a distribution regularizer in the objective function, for example in VAEs, the KL divergence between the prior distribution and the approximate posterior distribution. To do so, we first infer the distribution parameter λ , and then the proposed GENGS can be utilized by computing the PMF values π explicitly. However, we can instead directly infer the PMF vector π with the softmax function, not through the distribution parameter λ , as in Algorithm 3. We name the two cases as explicit inference and implicit inference (GENGS IMP), respectively, and GENGS IMP becomes possible by truncating distribution due to the finiteness of the dimension in the softmax function with the neural networks. Although the implicit inference cannot directly infer the distribution parameter λ , we found that loosening the posterior shape leads to a significant performance gain in our VAE examples.

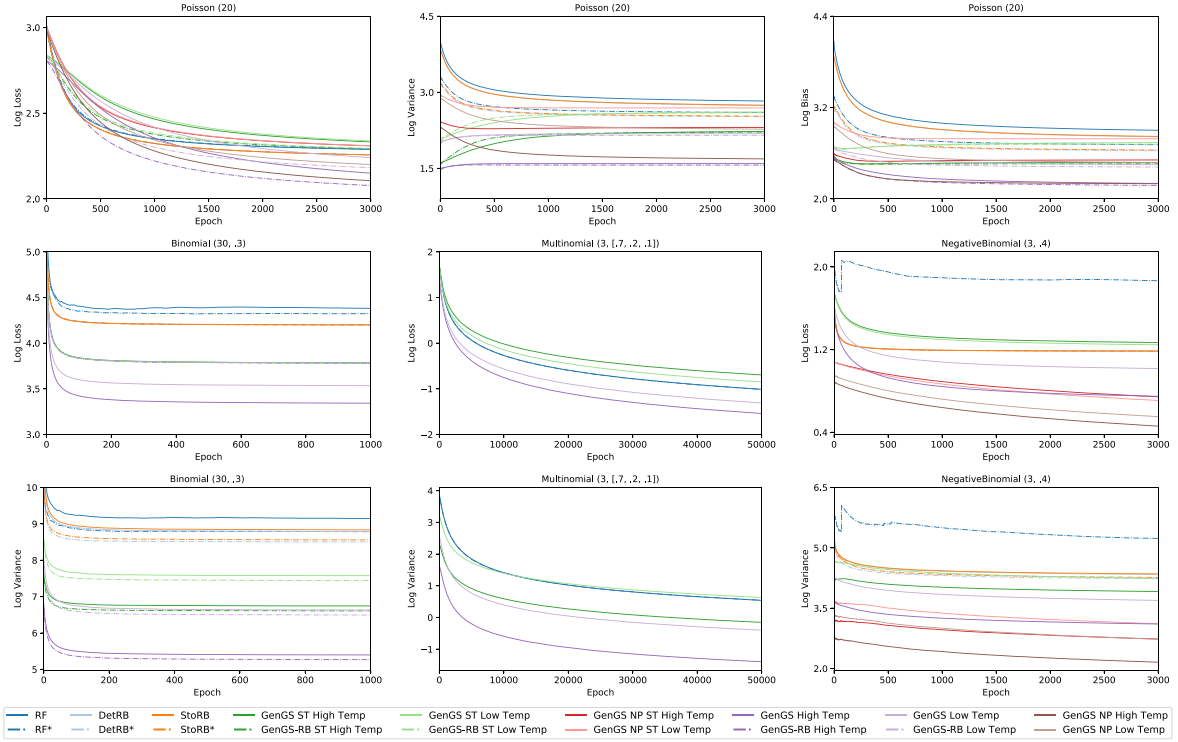


Fig. 3. Synthetic example performance curves in log scale: (Top Row) losses, variances, and biases of gradients for Poisson; (Middle Row) losses for Binomial, Multinomial, and NegativeBinomial; (Bottom Row) variances of gradients for Binomial, Multinomial, and NegativeBinomial. We utilize the cumulative average for smoothing the curves following from Kool et al. [25].

Discretization. Straight-Through (ST) method [12,26] can be applied to GENGS for drawing discrete samples. This variation draws discrete samples in the feed-forward step but utilizes the estimated gradient from the continuously relaxed and reparameterized GENGS in the back-propagation step. However, as in the original GS with the ST technique, GENGS with ST results in significant performance degradation in our synthetic examples.

5. Related work

We enumerate stochastic gradient estimators which can be utilized for generic discrete random variables. RF denotes the basic REINFORCE [21]. NVIL [22] utilizes a neural network to introduce the optimal control variate. MuPROP [23] utilizes the first-order Taylor expansion on the objective function as a control variate. VIMCO [27] is designed as a multi-sample gradient estimator. REBAR [14] and RELAX [15] utilize reparameterization tricks for constructing the control variate. DetRB [24] uses the weighted value of the fixed gradients from m -selected categories and the estimated gradients from the remaining categories with respect to their odds to reduce the variance. The idea of StoRB [25] is essentially the same as that of DetRB, but StoRB randomly selects the categories at each step. Kool et al. [25] also suggested UNORD, utilizing samples without replacements. Hence, DetRB, StoRB, and UNORD can be considered multi-sample gradient estimators. The * symbol denotes a built-in control variate introduced in the work of Kool et al. [25]. IRG [28] is an alternative to the GS, and its variations can be applied to approximate the PMF, including the Poisson, etc. However, IRG does not infer the distribution parameter as GENGS IMP does, and the paper mainly focuses on replacing the GS rather than reparameterizing the general class of the discrete random variables.

Algorithm 3 GENGS IMP: GENGS with implicit PMF.

- 1: **Input:** Distribution $D(\hat{\lambda})$ with inferred $\hat{\lambda}$, finite possible outcome vector c within truncation range R , temperature τ
- 2: $\hat{\pi} = \text{softmax}(\hat{\lambda})$
- 3: Compute $z = \text{GENGS}(\hat{\pi}, \tau)$
- 4: **return** z
- 5: Further optimize $\hat{\lambda}$ with Proposition 6

6. Experiment

6.1. Synthetic example

Experimental setting. This experiment expands the toy experiments from Tucker et al. [14], Grathwohl et al. [15] to diverse discrete distributions. We optimize the objective function $\mathbb{E}_{z \sim p(z|\lambda)} [\sum_{i=1}^k (z_i - t_i)^2]$ with respect to parameter λ for fixed constants $t_1, \dots, t_k$. Here, we set distribution $p(z|\lambda)$ as Poisson(λ), Binomial(20, λ), Multinomial(3, λ), and NegativeBinomial(3, λ). For the Poisson and the negative binomial, which have infinite support, we also applied GENGS NP described in Algorithm 2 to search the full range of support without pre-defined truncation ranges. Since the GENGSes are basically a single-sample gradient estimator, we utilize a single sample of z for the fundamental comparison among the gradient estimators in the toy example.

Experimental result. Fig. 3 compares the log-loss and the log-variance of estimated gradients from various estimators. The log-loss needs to be minimized with the estimated gradient value in the learning process by back-propagation. Additionally, the log-variance requires being minimized to maintain the consistency of the gradients so that the gradient descent can be efficient. The GENGSes show the best log-loss and the best log-variance if the GENGSes keep the continuous relaxation

Table 1

Training negative ELBO on MNIST and OMNIGLOT datasets. The lower is better for the negative ELBO, and the best and the second best results are marked in **bold** and underline, respectively. The symbol “—” indicates no convergence.

MNIST	Single-Sample Gradient Estimators								Multi-Sample (10) Gradient Estimators					
	RF*	NVIL	MuPROP	REBAR	RELAX	GENGS	GENGS IMP	GENGS NP	VIMCO	StoRB*	UNORD	GENGS	GENGS IMP	GENGS NP
Pois(2)	122.81 $\pm_{2.41}$	129.34 $\pm_{4.72}$	125.43 $\pm_{2.27}$	123.44 $\pm_{2.54}$	122.71 $\pm_{1.92}$	<u>103.18</u> $\pm_{0.92}$	96.04 $\pm_{1.44}$	104.98 $\pm_{1.19}$	122.13 $\pm_{3.02}$	122.71 $\pm_{3.83}$	130.95 $\pm_{4.66}$	<u>99.28</u> $\pm_{1.36}$	94.78 $\pm_{1.17}$	102.59 $\pm_{1.32}$
Pois(3)	123.12 $\pm_{2.21}$	130.24 $\pm_{3.32}$	125.92 $\pm_{1.81}$	120.62 $\pm_{2.31}$	119.84 $\pm_{2.18}$	<u>105.15</u> $\pm_{1.71}$	96.01 $\pm_{1.27}$	<u>104.25</u> $\pm_{1.56}$	120.93 $\pm_{2.48}$	121.92 $\pm_{3.12}$	119.97 $\pm_{5.38}$	<u>100.17</u> $\pm_{1.73}$	94.57 $\pm_{1.35}$	105.51 $\pm_{1.80}$
Geom(.25)	127.90 $\pm_{1.97}$	135.90 $\pm_{2.38}$	137.90 $\pm_{2.14}$	135.12 $\pm_{2.74}$	136.80 $\pm_{3.06}$	<u>98.43</u> $\pm_{0.81}$	92.52 $\pm_{1.62}$	<u>101.77</u> $\pm_{1.47}$	126.46 $\pm_{2.22}$	130.96 $\pm_{3.36}$	—	<u>92.59</u> $\pm_{1.07}$	92.06 $\pm_{0.78}$	100.34 $\pm_{1.77}$
Geom(.5)	129.20 $\pm_{2.03}$	138.47 $\pm_{2.30}$	136.40 $\pm_{1.78}$	138.37 $\pm_{2.98}$	139.41 $\pm_{3.59}$	<u>100.92</u> $\pm_{1.24}$	93.81 $\pm_{1.60}$	<u>97.29</u> $\pm_{1.89}$	128.71 $\pm_{2.81}$	134.10 $\pm_{4.65}$	—	<u>98.27</u> $\pm_{1.22}$	91.46 $\pm_{0.95}$	<u>96.44</u> $\pm_{1.38}$
NB(3,.5)	116.67 $\pm_{5.97}$	119.28 $\pm_{7.80}$	131.96 $\pm_{6.49}$	—	—	<u>98.58</u> $\pm_{1.27}$	94.52 $\pm_{1.52}$	<u>107.14</u> $\pm_{2.14}$	112.11 $\pm_{3.19}$	110.52 $\pm_{2.21}$	—	<u>96.83</u> $\pm_{1.63}$	93.66 $\pm_{1.03}$	99.85 $\pm_{1.91}$
NB(5,.3)	130.03 $\pm_{3.99}$	133.44 $\pm_{4.27}$	144.05 $\pm_{8.15}$	—	—	<u>100.88</u> $\pm_{2.35}$	95.37 $\pm_{1.43}$	<u>110.41</u> $\pm_{2.81}$	121.29 $\pm_{2.93}$	127.32 $\pm_{4.79}$	—	<u>97.29</u> $\pm_{1.87}$	94.29 $\pm_{1.28}$	105.52 $\pm_{2.53}$
OMNIGLOT	Single-Sample Gradient Estimators								Multi-Sample (10) Gradient Estimators					
	RF*	NVIL	MuPROP	REBAR	RELAX	GENGS	GENGS IMP	GENGS NP	VIMCO	StoRB*	UNORD	GENGS	GENGS IMP	GENGS NP
Pois(2)	139.47 $\pm_{3.29}$	148.01 $\pm_{4.19}$	142.95 $\pm_{1.32}$	138.12 $\pm_{3.26}$	137.56 $\pm_{2.94}$	127.89 $\pm_{1.44}$	118.17 $\pm_{2.22}$	<u>123.40</u> $\pm_{1.71}$	138.21 $\pm_{3.29}$	137.87 $\pm_{4.31}$	146.60 $\pm_{4.97}$	<u>120.42</u> $\pm_{2.21}$	116.92 $\pm_{1.52}$	121.81 $\pm_{2.47}$
Pois(3)	140.54 $\pm_{2.36}$	148.13 $\pm_{3.98}$	143.85 $\pm_{1.54}$	137.92 $\pm_{3.07}$	137.42 $\pm_{2.96}$	131.53 $\pm_{1.76}$	119.15 $\pm_{1.92}$	<u>126.03</u> $\pm_{2.16}$	138.84 $\pm_{2.69}$	139.89 $\pm_{3.98}$	147.18 $\pm_{5.23}$	<u>121.93</u> $\pm_{2.49}$	117.71 $\pm_{1.28}$	124.71 $\pm_{2.16}$
Geom(.25)	142.68 $\pm_{2.96}$	153.69 $\pm_{2.52}$	152.17 $\pm_{1.77}$	146.78 $\pm_{3.62}$	148.91 $\pm_{4.03}$	<u>115.23</u> $\pm_{2.00}$	107.79 $\pm_{2.84}$	<u>116.75</u> $\pm_{1.90}$	142.42 $\pm_{3.61}$	142.54 $\pm_{3.21}$	—	<u>113.49</u> $\pm_{1.95}$	105.38 $\pm_{1.13}$	115.69 $\pm_{2.03}$
Geom(.5)	142.70 $\pm_{1.77}$	153.20 $\pm_{1.49}$	149.76 $\pm_{2.19}$	149.63 $\pm_{3.49}$	151.97 $\pm_{3.90}$	115.14 $\pm_{2.43}$	108.48 $\pm_{2.78}$	<u>112.76</u> $\pm_{2.57}$	141.52 $\pm_{3.38}$	140.68 $\pm_{4.61}$	—	<u>108.91</u> $\pm_{1.89}$	105.97 $\pm_{1.30}$	112.41 $\pm_{1.85}$
NB(3,.5)	141.44 $\pm_{2.20}$	144.44 $\pm_{2.78}$	147.78 $\pm_{4.19}$	—	—	<u>118.57</u> $\pm_{2.71}$	117.02 $\pm_{2.18}$	<u>123.96</u> $\pm_{2.38}$	141.22 $\pm_{3.42}$	128.66 $\pm_{3.88}$	—	<u>116.55</u> $\pm_{2.23}$	114.20 $\pm_{1.61}$	120.92 $\pm_{2.34}$
NB(5,.3)	145.16 $\pm_{3.83}$	159.40 $\pm_{5.13}$	152.81 $\pm_{3.34}$	—	—	<u>119.57</u> $\pm_{2.02}$	117.54 $\pm_{2.76}$	<u>127.46</u> $\pm_{3.51}$	148.89 $\pm_{4.51}$	—	—	<u>118.37</u> $\pm_{2.51}$	114.77 $\pm_{1.47}$	124.98 $\pm_{2.95}$

of the modeled discrete random variable. For the Poisson, the exact gradient can be computed in closed-form, and the GENGSes show the lowest bias among all gradient estimators.

6.2. Synthetic experiment on DGMs

Experimental setting. To test the performance of the gradient estimators in DGMs, we adopt VAE in Section 2.2, which is one of the representative DGMs. We follow the VAE experiment scheme of Figurnov et al. [8], which utilizes various prior distributions. In our discrete case, we utilize the Poisson (Pois), the geometric (Geom), and the negative binomial (NB) distributions as the *latent factor count*. Note that the purpose of the VAE experiments is *not to compare the performance across various prior distributions*, such as the categorical or the Gaussian, but *to compare the performance across gradient estimators within the same prior distributions*. The VAE is considered a more challenging task than the synthetic example since (1) this task requires computing the gradients of the encoder network parameters through the latent distribution parameter λ ; and (2) each stochastic gradient of the latent dimension affects every encoder parameter since we are utilizing the fully-connected layers. Hence, a single poorly estimated gradient of the latent distribution parameter λ could harm the learning of encoder parameters, so the VAE experiment can dynamically show the performance of the gradient estimators. The VAEs optimize the ELBO, described in Eq. (4), which consists of the reconstruction part and the KL term. In the GENGS variants, by truncating the original distribution, the KL divergence between the approximate posterior and the prior distributions can be represented as the KL divergence between categorical distributions, as stated in Proposition 6. For the fair comparison, we separately compare the performance of the single-sample and multi-sample estimators with 10 samples. For the GENGS variants, we anneal the temperature from 1. to .1, and set truncation levels 12 for Pois(2); 15 for Pois(3); 25 for Geom(.25); 15 for Geom(.5); 30 for NB(3,.5); and 30 for NB(5,.3).

Proposition 6. Assume two truncated discrete distributions $X \sim TD(\lambda, n)$ and $Y \sim TD(\hat{\lambda}, n)$ where $\pi_k = P(X = k)$ and $\hat{\pi}_k = P(Y = k)$. Then, the KL divergence between X and Y can be represented in the KL divergence between the categorical distributions where $KL(Y \parallel X) = KL(\text{Categorical}(\hat{\pi}) \parallel \text{Categorical}(\pi))$.

Proof.

$$\begin{aligned}
 KL(Y \parallel X) &= \sum_k P(Y = k) \log \left(\frac{P(Y = k)}{P(X = k)} \right) \\
 &= \sum_k \hat{\pi}_k \log \left(\frac{\hat{\pi}_k}{\pi_k} \right) \\
 &= KL(\text{Categorical}(\hat{\pi}) \parallel \text{Categorical}(\pi)) \quad \square
 \end{aligned}$$

Table 2

NVPDEF test perplexity on 20NewsGroups and RCV1. (m) in the model name indicates that m samples are used for estimating the gradients. The lower is better for the perplexity, and the best results are marked in **bold**. The symbol “—” indicates no convergence.

	1-STACKED (Dim.)		2-STACKED (Dim.)	
	20News (50)	RCV1 (200)	20News (50)	RCV1 (200)
RF	1227 $\pm_{33.4}$	1182 $\pm_{27.8}$	RF(10)	—
RF*	944 $\pm_{17.7}$	708 $\pm_{15.6}$	RF*(10)	1221 $\pm_{19.6}$
NVIL	1077 $\pm_{22.5}$	1191 $\pm_{25.8}$	NVIL(10)	—
MuPROP	1045 $\pm_{20.1}$	935 $\pm_{19.3}$	MuPROP(10)	—
VIMCO(10)	958 $\pm_{17.1}$	741 $\pm_{16.2}$	VIMCO(20)	—
REBAR	934 $\pm_{27.6}$	716 $\pm_{25.9}$	REBAR(10)	1136 $\pm_{26.3}$
RELAX	932 $\pm_{31.1}$	717 $\pm_{29.4}$	RELAX(10)	1116 $\pm_{23.1}$
StoRB*(10)	919 $\pm_{16.2}$	701 $\pm_{14.7}$	StoRB*(20)	1094 $\pm_{35.7}$
UNORD(10)	1206 $\pm_{24.3}$	1188 $\pm_{22.1}$	UNORD(20)	—
GENGS	759 $\pm_{13.1}$	562 $\pm_{11.5}$	GENGS(10)	783 $\pm_{17.6}$
				576 $\pm_{18.8}$

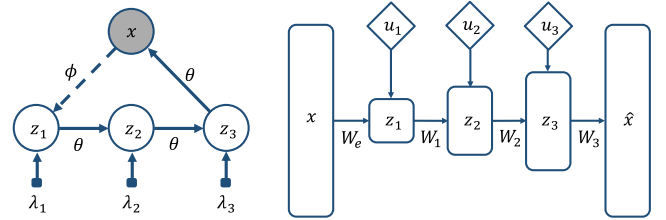


Fig. 4. (Left) A graphical notation of NVPDEF with the generative (θ) and inference (ϕ) processes. (Right) A neural network view of NVPDEF: diamond nodes indicate the auxiliary random variable for the reparameterization trick.

Experimental result. To compare the estimators from the optimization perspective, Table 1 presents the training negative ELBO on MNIST and OMNIGLOT for the VAE experiments. DETRB, DETRB*, and StoRB are not listed since they hardly lead to the optimal point. The variants of GENGS showed the lowest negative ELBO in general for both the single and the multi cases. Additionally, loosening the PMF condition (i.e., the implicit inference) led to better optimal points. The empirical reason why GENGS IMP is better than the explicit version is that the posterior PMF shape and support in the implicit case are thinner. Hence, the implicit distribution has a lower variance than the explicit one and draws consistent samples that lead to better-trained neural network parameters, although it loses the original PMF shape. However, it should be noted that the proposed GENGS IMP might ignore the original PMF shape. Therefore, GENGS IMP might be carefully utilized when the original PMF shape needs to be maintained. Even so, GENGS or GENGS NP leads to the second best negative ELBO, which validates the superiority of the proposed stochastic gradient estimator.

6.3. Application on topic model

Experimental setting. This experiment shows the practical usage of discrete distribution with GENGS in deep generative topic modeling. The authors of *Deep Exponential Families* (DEFs) [18] utilized the exponential family on the stacked latent layers. We focus on the Poisson DEF, which assumes the Poisson latent layers to capture the count of latent super-topics and sub-topics. We convert the Poisson DEF into a neural variational form, which resembles neural variational document model [29], namely neural variational Poisson deep exponential family (NVPDEF).

The generative and the inference process of NVPDEF are

$$z_k \sim \text{Pois}(\lambda_{k-1}) \text{ for } k = 1, \dots, K, \text{ and } x \sim \text{MLR}(\lambda_K), \text{ and} \\ \hat{\lambda}_0 = \text{MLP}(x), \text{ and } \hat{\lambda}_k = W_k \hat{\lambda}_{k-1} \text{ for } k = 1, \dots, K,$$

respectively, where MLR stands for multinomial logistic regression from Miao et al. [29]. Each $z_k \sim \text{Pois}(\lambda_{k-1})$ represents the count distribution of sub-topics from the super-topic, which is approximated by posterior $q(z_k|z_{k-1}) = \text{Pois}(\hat{\lambda}_k)$. Each component of W_k , $w_{k,i,j}$ is positive, and $w_{k,i,j}$ captures the positive weight of the relationship between super-topic i of the k th layer and sub-topic j of the $(k+1)$ th layer. Finally, NVPDEF optimizes

$$\mathcal{L} = \mathbb{E}_{q(z_{1:K})} [\log p(x|z_{1:K})] - \sum_{k=1}^K \text{KL}(q(z_k|z_{k-1}) \parallel p(z_k)), \quad (6)$$

where $z_0 = x$ for simplifying the equation. Fig. 4 shows the graphical notation and its corresponding neural network structure. Since the NVPDEF input of the bag-of-words form is sparser than the input of the VAE case, the experiment on the topic model application could extremely demonstrate the optimization stability of the stochastic gradient estimators.

For the single-stacked version of NVPDEF, we set $\lambda_1 = .75$ with truncation level 15 and fix the temperature as $\tau = .5$. For the multi-stacked version of NVPDEF, we set $\lambda_1 = 1.1, \lambda_2 = 1.$ with truncation level 15. To have better chances of learning during the training period in the multi-case, we utilize temperature annealing from $\tau = 3.$ to $\tau = .5$ and multiple samples on the latent layers for the stable optimization of consecutive random sampling. We compare the estimators by perplexity $\exp(-\frac{1}{D} \sum_d \frac{\log p(d)}{N_d})$ where N_d is the number of words in document d , and D is the total number of documents.

Experimental result. We enumerate the results of NVPDEF with various gradient estimators in Table 2, and we confirmed that GENGS gives the lowest perplexity with 20NewsGroups and RCV1, which is aligned with the VAE results. Especially, the multi-stacked RCV1 case extremely shows the optimization performance of the gradient estimators, where many of the gradient estimators fail to reach the optimal point, and GENGS gives the lowest perplexity.

7. Conclusion

This paper suggests variants of GENGS, a generalized version of the Gumbel-Softmax gradient estimator. GENGS utilizes the truncation of distribution support and the linear transformation to generalize the Gumbel-Softmax trick, specialized for reparameterizing categorical random variables. As a result, the proposed GENGS theoretically extends the range of reparameterizable discrete distributions to general discrete distributions. Empirically, the synthetic analysis and the VAE experiment show the efficacy of GENGS, and the topic model application presents the practical usage of GENGS. The proposed GENGS can be simply implemented, and the experimental result shows that its variants lead to better optimization results compared to the existing stochastic gradient estimators. With the generalization, we expect that GENGS can diversify the options of distributions in the deep generative model community. While deep generative models can now utilize generic discrete distributions with the proposed GENGS, the characteristics of each discrete random variable have not been studied. Since the feature and usage of each discrete random variable are also crucial in deep generative modeling, we leave these as future work.

CRediT authorship contribution statement

Weonyoung Joo: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Dongjun Kim:** Formal analysis, Methodology. **Seungjae Shin:** Formal analysis, Software. **Il-Chul Moon:** Funding acquisition, Supervision, Resources, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by (1) the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00334477); (2) the EWha Womans University Research Grant of 2022; and (3) the IITP (Institute of Information & Communications Technology Planning & Evaluation) - ITRC (Information Technology Research Center) grant funded by the Korea government (Ministry of Science and ICT) (IITP-2025-RS-2024-00437268).

Data availability

The code/data will be made available later.

References

- [1] D.P. Kingma, M. Welling, Auto-encoding variational bayes, *Int. Conf. Learn. Represent.* (2014).
- [2] S. Jiang, Y. Chen, J. Yang, C. Zhang, T. Zhao, Mixture variational autoencoders, *Pattern Recognit. Lett.* 128 (2019) 263–269.
- [3] T. Song, J. Sun, X. Liu, J. Song, W. Peng, Topic-word-constrained sentence generation with variational autoencoder, *Pattern Recognit. Lett.* 160 (2022) 148–154.
- [4] M. Xu, M. Quiroz, R. Kohn, S.A. Sisson, Variance reduction properties of the reparameterization trick, *Int. Conf. Artif. Intell. Stat.* (2019).
- [5] D.P. Kingma, M. Welling, Efficient gradient-based inference through transformations between bayes nets and neural nets, *Int. Conf. Mach. Learn.* (2014).
- [6] E. Nalisnick, P. Smyth, Stick-breaking variational autoencoders, *Int. Conf. Learn. Represent.* (2017).
- [7] W. Joo, W. Lee, S. Park, I.C. Moon, Dirichlet variational autoencoder, *Pattern Recognit.* 107, 107514 (2020).
- [8] M. Figurnov, S. Mohamed, A. Mnih, Implicit reparameterization gradients, *Adv. Neural Inf. Process. Syst.* (2018).
- [9] M. Jankowiak, F. Obermeyer, Pathwise derivatives beyond the reparameterization trick, *Int. Conf. Mach. Learn.* (2018).
- [10] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al., TensorFlow: A system for large-scale machine learning, *USENIX Symp. Oper. Syst. Des. Implement.* (2016).
- [11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, *Adv. Neural Inf. Process. Syst.* (2019).
- [12] E. Jang, S. Gu, B. Poole, Categorical reparameterization with Gumbel-softmax, *Int. Conf. Learn. Represent.* (2017).
- [13] C.J. Maddison, A. Mnih, Y.W. Teh, The concrete distribution: A continuous relaxation of discrete random variables, *Int. Conf. Learn. Represent.* (2017).
- [14] G. Tucker, A. Mnih, C.J. Maddison, J. Lawson, J. Sohl-Dickstein, REBAR: Low-variance, unbiased gradient estimates for discrete latent variable models, *Adv. Neural Inf. Process. Syst.* (2017).
- [15] W. Grathwohl, D. Choi, Y. Wu, G. Roeder, D. Duvenaud, Backpropagation through the void: Optimizing control variates for black-box gradient estimation, *Int. Conf. Learn. Represent.* (2017).
- [16] A. Gadetsky, K. Struminsky, C. Robinson, N. Quadrianto, D.P. Vetrov, Low-variance black-box gradient estimates for the Plackett-Luce distribution, *Assoc. Adv. Artif. Intell.* (2020).

- [17] M.B. Paulus, D. Choi, D. Tarlow, A. Krause, C.J. Maddison, Gradient estimation with stochastic softmax tricks, *Adv. Neural Inf. Process. Syst.* (2020).
- [18] R. Ranganath, L. Tang, L. Charlin, D. Blei, Deep exponential families, *Artif. Intell. Stat.* (2015).
- [19] R. Ranganath, A. Perotte, N. Elhadad, D. Blei, Deep survival analysis, in: *Machine Learning for Healthcare Conference*, PMLR, 2016.
- [20] J. Wu, Y. Rao, Z. Zhang, H. Xie, Q. Li, F.L. Wang, Z. Chen, Neural mixed counting models for dispersed topic discovery, *Annu. Meet. Assoc. Comput. Linguist.* (2020).
- [21] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* 8 (3-4) (1992) 229–256.
- [22] A. Mnih, K. Gregor, Neural variational inference and learning in belief networks, *Int. Conf. Mach. Learn.* (2014).
- [23] S. Gu, S. Levine, I. Sutskever, A. Mnih, Muprop: Unbiased backpropagation for stochastic neural networks, *Int. Conf. Learn. Represent.* (2016).
- [24] R. Liu, J. Regier, N. Tripuraneni, M.I. Jordan, J. McAuliffe, Rao-blackwellized stochastic gradients for discrete distributions, *Int. Conf. Mach. Learn.* (2019).
- [25] W. Kool, H. van Hoof, M. Welling, Estimating gradients for discrete random variables by sampling without replacement, *Int. Conf. Learn. Represent.* (2020).
- [26] Y. Bengio, N. Leonard, A. Courville, Estimating or propagating gradients through stochastic neurons for conditional computation, 2013, *ArXiv Preprint ArXiv:1308.3432*.
- [27] A. Mnih, D.J. Rezende, Variational inference for Monte Carlo objectives, *Int. Conf. Mach. Learn.* (2016).
- [28] A. Potapczynski, G. Loaiza-Ganem, J.P. Cunningham, Invertible Gaussian reparameterization: Revisiting the Gumbel-softmax, *Adv. Neural Inf. Process. Syst.* (2020).
- [29] Y. Miao, L. Yu, P. Blunsom, Neural variational inference for text processing, *Int. Conf. Mach. Learn.* (2016).